Studio Nx

Backend(FastAPI/Python) 36 — to_me_response()

2026-10-12

Backend(FastAPI/Python) 36 — `to_me_response()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-08 회원 정보: 내 정보 응답 만들기 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	35 `profile_image_url()` · 37 `update_me()`

1. 하는 일

DB의 회원 한 행(`User`)을 화면에 보낼 **내 정보 응답(`MeResponse`)**으로 바꿉니다. 미리 정해 둔 7칸만 담습니다.

```
{
  "user_id": 85,
  "email": "demo@example.com",
  "nickname": "demo_user",
  "role": "USER",
  "profile_image_url": "/api/v1/profile-images/3f9c….jpg",
  "created_at": "2026-10-07T01:12:30",
  "last_login_at": "2026-10-07T09:30:00"
}
```

담지 않는 칸 (User에는 있음)	이유
`password_hash`	해시라도 밖으로 나가면 안 됩니다. 무작위 대입으로 원래 비밀번호를 찾아내는 데 쓰일 수 있습니다
`status`, `deleted_at`	이 응답은 로그인한 **활동 회원**만 받습니다 (06번이 이미 막습니다)
`privacy_agreed_at`, `privacy_version`, `storage_*` 등	회원 정보 화면이 쓰지 않는 칸입니다. 용량은 P1-16 대시보드 응답이 따로 보내 줍니다

2. 언제 쓰나

쓰는 곳 (P1-08 라우터)	
`GET /users/me` (41)	내 정보를 보낼 때
`PATCH /users/me` (41)	닉네임을 바꾼 뒤 바뀐 정보를 보낼 때
`POST`, `DELETE /users/me/profile-image` (42)	사진을 바꾸거나 지운 뒤 바뀐 정보(새 `profile_image_url`)를 보낼 때

화면에서는 `userinfo.js`의 `loadMe`(P1-10)와 `mypage.js`가 씁니다.

응답에 `user_id`를 넣어도 괜찮습니다. 이 응답은 **토큰을 가진 본인**에게만 가기 때문입니다. 탈퇴 추적 방지 ③은 "남의 번호로 조회하는 공개 API"를 막는 규칙입니다.

3. 코드

import (파일 맨 위: 35번과 같고, 새로 추가한 것은 없습니다)

```
from pathlib import Path, PurePosixPath

from fastapi import UploadFile
from sqlalchemy.orm import Session

from app.models import User
from app.schemas.user import MeResponse, MeUpdateRequest
```

이 함수는 **`User`, `MeResponse`**와 35번 `profile_image_url`을 사용합니다.

함수

```
def to_me_response(user: User) -> MeResponse:
    return MeResponse(
        user_id=user.user_id,
        email=user.email,
        nickname=user.nickname,
        role=user.role,
        profile_image_url=profile_image_url(user),
        created_at=user.created_at,
        last_login_at=user.last_login_at,
    )
```

문법 풀이

import 문법	뜻
`from app.schemas.user import MeResponse`	템플릿에 있던 응답 모양입니다 (pydantic `BaseModel`)

식	뜻
`MeResponse(user_id=…, …)`	이름을 붙인 인자(키워드 인자)로 칸마다 값을 넣어 만듭니다. 칸이 빠졌거나 형식이 틀리면 pydantic이 바로 오류를 냅니다
`role=user.role`	DB 값은 `UserRole.USER`(StrEnum)입니다. `StrEnum`은 문자열이기도 해서 응답 칸 `str`에 그대로 들어가고, JSON에는 `"USER"`로 나갑니다
`created_at` (`datetime`)	FastAPI가 JSON으로 바꿀 때 `"2026-10-07T01:12:30"` 모양의 문자열로 만듭니다

4. 흐름

1. 회원의 칸 6개를 그대로 옮깁니다.
2. 사진은 35번으로 주소를 만듭니다.
3. 응답 객체를 돌려줍니다.

5. 왜 이렇게

선택	이유
칸을 **하나씩** 적음	`MeResponse.model_validate(user)`(ORM 객체에서 자동으로 꺼내기)를 쓸 수도 있습니다. 하지만 그러려면 스키마에 `from_attributes` 설정이 필요하고, `profile_image_url`은 DB 칸이 아니라 따로 계산해야 합니다. 하나씩 적어 두면 무엇이 나가는지 한눈에 보입니다

선택	이유
응답 칸을 스키마로 고정	나중에 `User`에 칸이 늘어나도 응답에 저절로 섞이지 않습니다. 시험이 "칸은 7개뿐"인지 확인합니다
DB를 다시 읽지 않음	부르는 쪽이 이미 최신 `User`를 가지고 있습니다 (수정한 뒤에는 37번과 38번이 `refresh`합니다)

6. 시험 (`tests/test_user_service.py`에 3개를 추가해서 모두 8개, 실제 MySQL로 실행)

시험	확인한 것
새 회원	7칸의 값이 DB와 같고, `role`은 `"USER"`, 사진과 마지막 로그인은 `None`이며, `created_at`은 DB가 채운 값입니다
응답 칸	정확히 7개입니다. JSON 안에 비밀번호 해시 문자열과 `password`가 없습니다
관리자, 사진, 로그인 기록	`role`은 `"ADMIN"`, `last_login_at`은 그대로, 사진 주소는 `/api/v1/profile-images/{32자}.png`입니다

전체 시험 242개가 통과했고, `ruff` 검사도 통과했습니다. 시험에서 만든 회원은 시험이 끝나면 되돌려집니다 (conftest의 savepoint).